



**UNIVERSITÉ  
DE GENÈVE**

BACHELOR'S THESIS

# Tree-Based Classification of Encrypted Network Traffic

**Author:** Manuel Speranza

**Supervisors:** Eduardo Solana, Pierre Leone

Spring 2025

# Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>2</b>  |
| <b>2</b> | <b>Material</b>                                                  | <b>4</b>  |
| 2.1      | Datasets . . . . .                                               | 4         |
| 2.1.1    | VPN-nonVPN dataset (ISCXVPN2016) . . . . .                       | 4         |
| 2.1.2    | VPN/Non-VPN Network Application Traffic Dataset (VNAT) . . . . . | 4         |
| 2.2      | Computation Environment . . . . .                                | 5         |
| 2.3      | Software Tool: EnTrAX . . . . .                                  | 5         |
| <b>3</b> | <b>Data Acquisition and Preprocessing</b>                        | <b>6</b>  |
| 3.1      | Packet Parsing and Flow Construction . . . . .                   | 6         |
| 3.2      | Feature Extraction from Flows . . . . .                          | 7         |
| 3.2.1    | Numeric Flow Features . . . . .                                  | 7         |
| 3.2.2    | Label Assignment . . . . .                                       | 9         |
| 3.3      | Dataset Overview and Correlation Analysis . . . . .              | 9         |
| <b>4</b> | <b>Tree Models</b>                                               | <b>12</b> |
| 4.1      | Decision Trees . . . . .                                         | 12        |
| 4.1.1    | Classification and Regression Tree (CART) . . . . .              | 12        |
| 4.1.2    | CART Results . . . . .                                           | 13        |
| 4.2      | Tree Ensembles . . . . .                                         | 14        |
| 4.2.1    | Random Forests . . . . .                                         | 14        |
| 4.2.2    | Random Forests Results . . . . .                                 | 15        |
| 4.3      | Boosting . . . . .                                               | 16        |
| 4.3.1    | Binary Boosting Algorithm . . . . .                              | 16        |
| 4.3.2    | Boosting CART Tree Construction . . . . .                        | 17        |
| 4.3.3    | Binary Boosting Results . . . . .                                | 18        |
| 4.4      | Multi-Class Boosting . . . . .                                   | 18        |
| 4.4.1    | Multi-Class Boosting Algorithm . . . . .                         | 18        |
| 4.4.2    | Multi-Class Boosting Results . . . . .                           | 19        |
| <b>5</b> | <b>Conclusion</b>                                                | <b>21</b> |
| <b>A</b> | <b>Appendix</b>                                                  | <b>22</b> |
| A.1      | .pcap and .pcapng . . . . .                                      | 22        |
| A.2      | TCP RTT Calculation . . . . .                                    | 22        |
| A.3      | Correlation Matrix . . . . .                                     | 22        |
| A.4      | Gini impurity . . . . .                                          | 23        |
| A.5      | Information Gain with Weighted Gini . . . . .                    | 23        |
| A.6      | Metrics . . . . .                                                | 23        |
| A.7      | Sigmoid Function . . . . .                                       | 24        |
| A.8      | Binary Cross-Entropy Loss . . . . .                              | 24        |
| A.9      | Gradient . . . . .                                               | 24        |
| A.10     | Hessian . . . . .                                                | 25        |
| A.11     | Softmax Function . . . . .                                       | 25        |

# 1 Introduction

Today, most Internet traffic is protected by strong end-to-end encryption. Protocols such as TLS 1.3 encrypt packet payloads, so anyone in the middle can no longer read the text, images, or commands inside. However, certain metadata remains exposed, such as packet sizes, timing, direction, IP/port headers, etc. The study of these can reveal surprisingly rich information about user activity without ever breaking the encryption itself.

Depending on the observer’s intent, this traffic metadata analysis can be used for beneficial monitoring or for privacy-invasive surveillance. For example, a network operator could use it to detect malicious activity, while an attacker could use it to infer sensitive information about users. Knowing the possibility of these dangers, it becomes important to understand how far this analysis can go and what is necessary for its implementation in terms of techniques and resources.

A wide range of analysis tasks is already possible on encrypted traffic, such as asset identification, privacy-leakage detection, and network characterization. The most common approach to these tasks is the use of algorithmic statistical modeling, classical machine learning, or deep learning. The choice between the techniques often depends on the volume of data, real-time requirements, and available compute resources. In general, more powerful models and greater computational capacity allow the extraction of more information from the data. The focus of this work is on *network characterization*, specifically the classification of encrypted network flows according to the application that generated them.

**Related work** In recent years, a large number of studies have explored encrypted traffic classification, leading to the development of various datasets containing different traffic types and applications. Among the most frequently used are **ISCXVPN2016** [1] and **VNAT** [2]. Datasets like ISCXVPN2016 are particularly valuable because they contain a broad range of labeled applications, making the classification task more representative of real-world scenarios.

Several approaches have been proposed for this task, ranging from traditional machine learning techniques, often based on decision trees [1] [3], to increasingly popular deep learning methods [4]. Deep learning models can reach very high accuracy and are capable of leveraging more complex traffic features, including packet payloads. However, such models typically require substantial computational resources, both in terms of processing power and memory, which may not always be available or practical in real-world deployments. Comparative studies, such as [3], have shown that both decision tree models and deep learning models can achieve strong classification performance.

Nonetheless, these studies sometimes use datasets with fewer classes, reducing the complexity of the classification problem. Additionally, given the growing body of work in this field, several surveys have been published to organize and analyze the different methods, datasets, and evaluation strategies used in encrypted traffic analysis and network classification [5, 6].

**Objectives of this work** The approach taken in this work follows the direction of Draper-Gil et al. [1], focusing on a lightweight pipeline that compute the features only from basic packet metadata, specifically packet sizes and timestamps. The goal is to keep computational requirements low while still achieving high classification performance. This makes the solution more accessible for a wider range of devices, especially in resource-constrained environments.

This work follows the complete pipeline required for encrypted-traffic classification:

- **Section 2** introduces the material used, the two reference datasets, the computing environment, and the custom CLI tool that automates every subsequent step.
- **Section 3** explains how raw packet-capture files are parsed into flows, and then the features computed, for finally obtaining a dataset ready for classification.
- **Section 4** evaluates a series of tree-based classifiers in increasing order of complexity until reaching a gradient-boosted tree model based on the XGBoost algorithm [7], a scalable and efficient gradient-boosted decision tree method that has shown strong performance in many classification tasks. It begins with a single CART baseline, moves to Random Forests, and then to gradient-boosted trees—first for the binary VPN task and finally for the full multi-class problem. Each model is trained on the and tested on the dataset obtained in section 3.

At every stage, the work tracks both accuracy and computational cost, showing how far basic metadata can be pushed while keeping the approach lightweight enough for real-world applications.

## 2 Material

### 2.1 Datasets

#### 2.1.1 VPN-nonVPN dataset (ISCXVPN2016)

The ISCXVPN2016 dataset, also known as the VPN-nonVPN dataset, was created to provide a comprehensive and diverse collection of real-world network traffic. It includes both regular and VPN-encrypted sessions across 14 traffic categories, such as web browsing, email, chat, streaming, file transfer, VoIP, and peer-to-peer (P2P) communication. Traffic was generated through various applications, including web browsers (Chrome, Firefox), email clients (Thunderbird), messaging platforms (Skype, Facebook, Hangouts, ICQ, AIM), streaming services (YouTube, Vimeo), file transfer protocols (SFTP, FTPS), and P2P clients (uTorrent, Transmission). The dataset consists of full packet captures (pcap format) and flow-based records (CSV format) generated using ISCXFlowMeter. The traffic collection process ensured precise labeling by isolating the execution of each application during data capture. VPN traffic was recorded using an external VPN service with OpenVPN in UDP mode. In total, the dataset comprises approximately 28 GB of captured network traffic, making it a valuable resource for research in network security, traffic classification, and encrypted traffic analysis. The dataset is used and described further in the research paper:[\[1\]](#)

#### 2.1.2 VPN/Non-VPN Network Application Traffic Dataset (VNAT)

The VPN/Non-VPN Network Application Traffic (VNAT) dataset, released by MIT Lincoln Laboratory, is designed to support research in encrypted traffic classification and network behavior analysis. It captures a wide range of traffic from both VPN and non-VPN environments, emulating real-world network usage patterns across multiple categories.

The dataset includes traffic from common applications and protocols such as web browsing, file sharing, video streaming, email, and messaging services. Data is collected from multiple operating systems and VPN clients, including OpenVPN and L2TP/IPsec, to ensure protocol diversity. Both encrypted (VPN) and plaintext (non-VPN) traffic sessions are recorded.

Traffic was captured in controlled environments to ensure accurate labeling, with packet-level data saved in pcap format. Annotations and metadata are provided to facilitate supervised machine learning tasks. The VNAT dataset serves as a valuable benchmark for evaluating classification models that aim to distinguish VPN from non-VPN traffic, even under protocol obfuscation or encryption.

The dataset is used and described further in the research paper:[\[2\]](#)

## 2.2 Computation Environment

All experiments were executed remotely on a university’s machine, accessed via SSH.

Table 1: Hardware and software specifications used for all experiments

| Component                | Specification                              |
|--------------------------|--------------------------------------------|
| CPU                      | AMD EPYC 9654 96-Core Processor            |
| Physical / Logical cores | 192 / 384                                  |
| Clock speed              | 1.5 GHz (base), 3.7 GHz (boost)            |
| RAM                      | 755 GB DDR4                                |
| Storage                  | 7 TB NVMe SSD (Samsung MZQL27T6HBLA-00A07) |
| Operating system         | Ubuntu 24.04.1 LTS                         |

## 2.3 Software Tool: EnTrAX

To make all experiments and subsequent analysis in this work possible, a CLI tool named **EnTrAX** was purposefully developed for this research project. **EnTrAX** automates the entire encrypted traffic classification pipeline, from raw packet capture files to the generation of trained machine learning models. The tool was developed in Python, making use of several libraries to handle data processing and model training effectively.

The source code is available on GitLab: <https://gitlab.unige.ch/Manuel.Speranza/entrax>.

### 3 Data Acquisition and Preprocessing

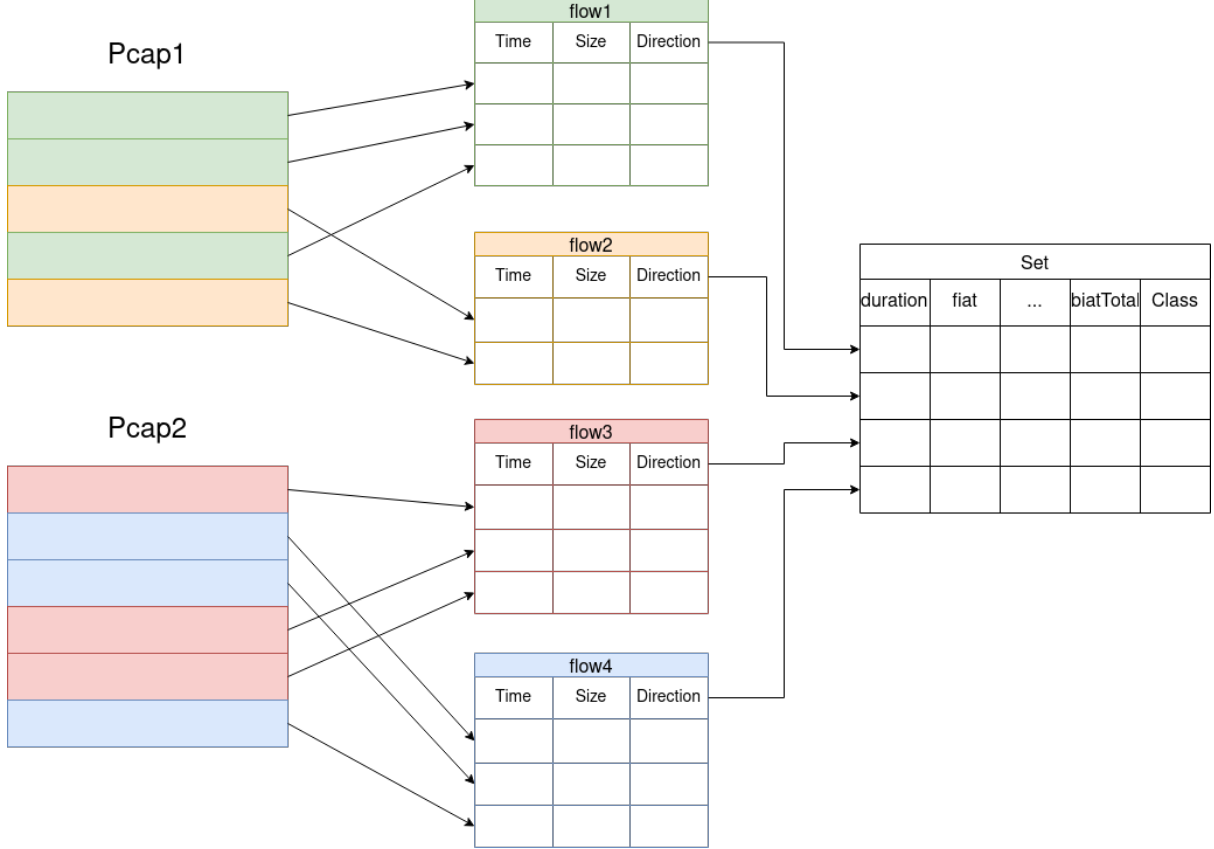


Figure 1: From capture file to CSV dataset: packets are split into flows and stored.

#### 3.1 Packet Parsing and Flow Construction

This work uses two datasets, **ISCXVPN2016** and **VNAT**. Both provide network traffic in .pcap or .pcapng format (see Appendix A.1). Each capture file contains traffic generated by a single application, which is noted in the file name.

To group packets into communication sessions, flows are constructed using the following standard five-tuple  $\langle IP_{src}, IP_{dst}, port_{src}, port_{dst}, protocol \rangle$ .

The first packet seen in the flow defines the source and destination roles, and the transport protocol is either TCP or UDP.

Packet parsing is performed within **EnTraX** using the **PyShark** library [8]. For each packet we keep only three fields: the timestamp, the packet length, and a binary flag for the direction (from source or destination). Restricting the feature set to these basic attributes reduces both processing time and memory use during classification.

## Extraction statistics

- **ISCXVPN2016**

- 140 capture files
- 25 976 474 packets
- 10 186 flows
- Processing time  $\approx 22$  h

- **VNAT**

- 165 capture files
- 38 275 373 packets
- 3 833 flows
- Processing time  $\approx 68$  h

## 3.2 Feature Extraction from Flows

### 3.2.1 Numeric Flow Features

A flow can be represented as a matrix with size  $n \times 3$ , where each row represents a packet and the columns correspond to its attributes. The matrix is defined as:

$$\mathbf{F}_{n \times 3} = \begin{bmatrix} t_1 & s_1 & d_1 \\ t_2 & s_2 & d_2 \\ \vdots & \vdots & \vdots \\ t_n & s_n & d_n \end{bmatrix}$$

Where:

- $t_i$  Time, in seconds, from the start of the flow until packet  $i$  is sent. It is stored as a floating-point value ( $t_0 = 0$ ).
- $s_i$  Size of packet  $i$  in bytes (integer).
- $d_i$  Direction flag for packet  $i$ . If the packet goes from the flow's source to its destination,  $d_i = \text{Forward (True)}$ ; otherwise  $d_i = \text{Backward (False)}$ .

With  $1 \leq i \leq n$

The matrix is saved in a CSV file, other column are present in the saved CSV, but they correspond to metadata, so they will not be used for creating the dataset

For each flow, we compute<sup>1</sup> various set of features, which will be used as predictors for the classification. Certain features describe the flow as a whole, while others are computed for each packet, for the last ones only some statistics are kept, such as mean, maximum, minimum, standard deviation, and total. Also, a difference is made between the forward and backward direction. The features are based on time and size of the packets. The following list summarizes the features computed for each flow:

---

<sup>1</sup>All computations were implemented in the CLI tools with extensive use of the NumPy [9] and pandas [10] libraries.



Table 2: Flow-based Network Traffic Features and Corresponding Field Names/Statistics

| Metric                                    | Description                                                                                           | Field name/Computed statistics                                                                                                |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Duration</b>                           | The total duration of the flow, defined as the timestamp of the last packet in the session ( $t_n$ ). | <code>duration</code>                                                                                                         |
| <b>Flow Inter-Arrival Time (FLOWIAT)</b>  | Time intervals between two consecutive packets in either direction.                                   | <code>flowiatMean</code> , <code>flowiatMax</code> , <code>flowiatMin</code> , <code>flowiatStd</code>                        |
| <b>Forward Inter-Arrival Time (FIAT)</b>  | Time intervals between two consecutive forward direction packets.                                     | <code>fiatMean</code> , <code>fiatMax</code> , <code>fiatMin</code> , <code>fiatStd</code> , <code>fiatTotal</code>           |
| <b>Backward Inter-Arrival Time (BIAT)</b> | Time intervals between two consecutive backward direction packets.                                    | <code>biatMean</code> , <code>biatMax</code> , <code>biatMin</code> , <code>biatStd</code> , <code>biatTotal</code>           |
| <b>Active Time</b>                        | Periods during which the flow is actively transmitting packets.                                       | <code>activeMean</code> , <code>activeMax</code> , <code>activeMin</code> , <code>activeStd</code> , <code>activeTotal</code> |
| <b>Idle Time</b>                          | Periods during which the flow is inactive.                                                            | <code>idleMean</code> , <code>idleMax</code> , <code>idleMin</code> , <code>idleStd</code> , <code>idleTotal</code>           |
| <b>Bytes per Second (BS)</b>              | Throughput in bytes per second, calculated as <code>PLTotal</code> divided by <code>duration</code> . | <code>bs</code>                                                                                                               |
| <b>Packets per Second (PS)</b>            | Packet rate, calculated as the total number of packets divided by <code>duration</code> .             | <code>ps</code>                                                                                                               |
| <b>Flow Packets Number (FLOWPN)</b>       | Number of packets in either direction.                                                                | <code>flowPN</code>                                                                                                           |
| <b>Forward Packets Number (FWDPN)</b>     | Number of packets transmitted in the forward direction.                                               | <code>fwdPN</code>                                                                                                            |
| <b>Backward Packets Number (BWDPN)</b>    | Number of packets transmitted in the backward direction.                                              | <code>bwdPN</code>                                                                                                            |
| <b>Flow Packet Length (FLOWPL)</b>        | Lengths of all packets in the flow, represented by vector $s$ .                                       | <code>flowPLMean</code> , <code>flowPLMax</code> , <code>flowPLMin</code> , <code>flowPLStd</code> , <code>flowPLTotal</code> |
| <b>Forward Packet Length (FWDPL)</b>      | Lengths of packets sent in the forward direction.                                                     | <code>fwdPLMean</code> , <code>fwdPLMax</code> , <code>fwdPLMin</code> , <code>fwdPLStd</code> , <code>fwdPLTotal</code>      |
| <b>Backward Packet Length (BWDPL)</b>     | Lengths of packets sent in the backward direction.                                                    | <code>bwdPLMean</code> , <code>bwdPLMax</code> , <code>bwdPLMin</code> , <code>bwdPLStd</code> , <code>bwdPLTotal</code>      |

**Active and Idle Periods** Defining active and idle periods is challenging due to the nature of the traffic and the protocol used. For UDP, a 15 s timeout is used to separate active and idle periods based on packet arrival gaps. For TCP, the round trip time (RTT) is approximated as follows. We start with an initial RTT of 15 s and, for each new packet, update it by

$$RTT_{i+1} = \alpha RTT_i + (1 - \alpha) M_i, \quad \alpha = \frac{7}{8},$$

where  $M_i$  is the time between a packet and the next packet received from the other endpoint. So the flow is considered as Idle if  $M_i > RTT_i$

This method only approximates the true RTT computation which should be measured from ACKs but parsing and matching ACKs with respective packets, would complicate data extraction without significantly improving the analysis results. (Appendix A.2)

### 3.2.2 Label Assignment

Once the numerical features have been computed, the next step is to assign a label to each flow. Because the dataset was captured in an isolated environment, the providers encoded the true class of each flow directly in the file name, for example, whether the traffic is VPN or non-VPN and which application generated it. So it is possible to recover the label simply by parsing the file name from which the flow was extracted, as follows<sup>2</sup>

| Filename Keyword        | Class Label     | VPN Class Label     |
|-------------------------|-----------------|---------------------|
| chat                    | CHAT            | VPN-CHAT            |
| email                   | MAIL            | VPN-MAIL            |
| voip, audio             | VOIP            | VPN-VOIP            |
| netflix, vimeo, youtube | VIDEO STREAMING | VPN-VIDEO STREAMING |
| video                   | VIDEOCALL       | VPN-VIDEOCALL       |
| ftps, sftp, scp, file   | FT              | VPN-FT              |

Table 3: Base and VPN-prefixed class labels for each filename keyword. The VPN version applies when the filename contains the substring "vpn".

## 3.3 Dataset Overview and Correlation Analysis

After feature extraction we obtain two flow tables, one per dataset. The **ISCXVPN2016** table contains 10 186 flows, while **VNAT** contains 3 833 flows. Each flow is described by 45 candidate predictors plus a class label.

For the classification experiments, we merge the two tables into a single dataset, and split it 80 %-20 % into training and test sets. We merge ISCXVPN2016 and VNAT into one dataset to enlarge the training set and to expose the model to traffic captured by different operators and under varying network conditions. This increases task difficulty but yields a more robust and practically relevant classifier.

Using all 45 predictors can slow training and may not improve accuracy if some features carry overlapping information. Therefore, the Pearson correlation between every pair of features (Appendix A.3) is checked, and highly redundant features are discarded prior to model fitting.

<sup>2</sup>The dataset also contains an *Audio Streaming* class and its VPN counterpart. Due to the very small sample size, it is excluded to avoid degrading model performance.

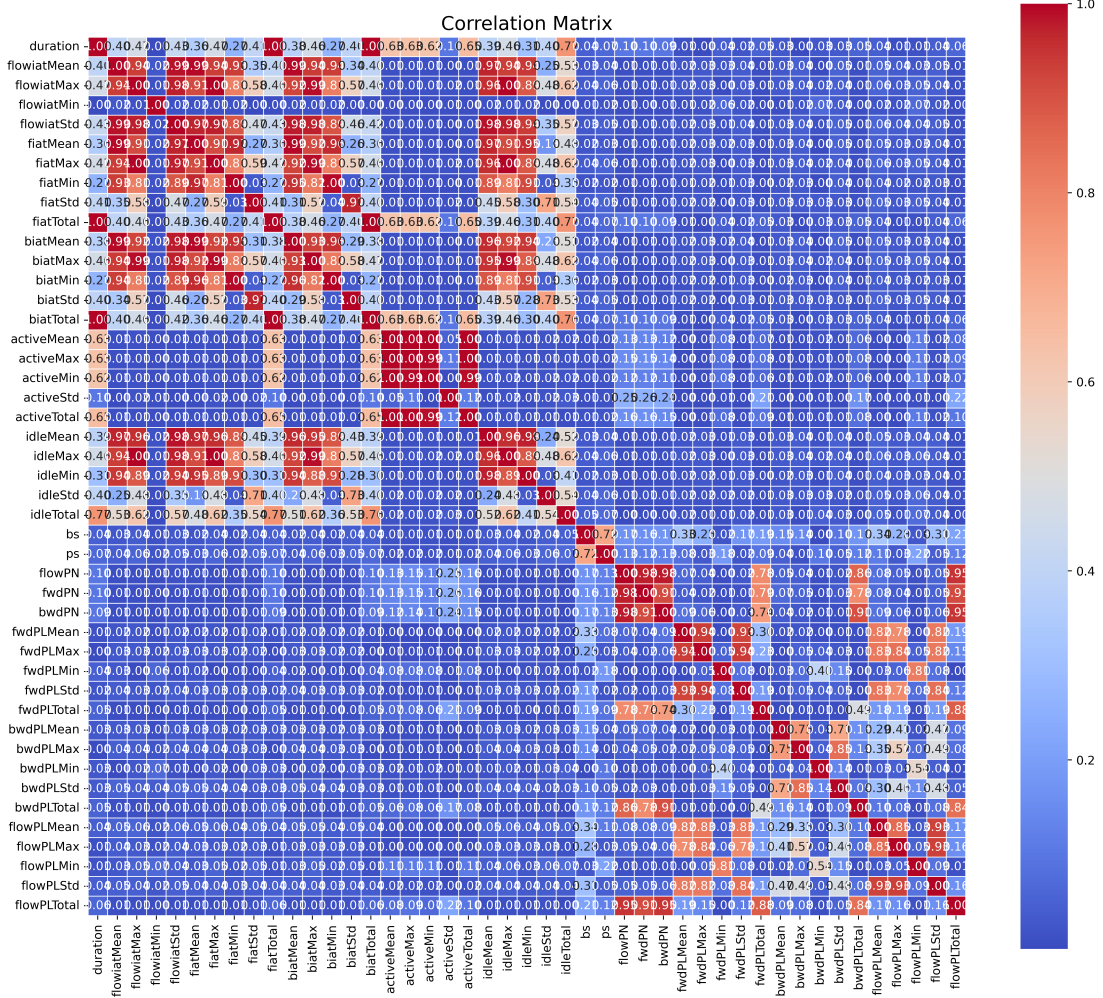


Figure 2: Pearson correlation matrix for the full feature set.

Figure 2 shows that high correlations occur mainly within the same measurement family, such as packets length statistics or time metrics, but different summaries of the same quantity (max, min, mean, standard deviation, total) are *not* always strongly correlated, so each still carries unique information.

To reduce redundancy and speed up model training, any feature whose absolute correlation with another exceeds 0.9 is dropped.

This removes 24 variables: flowiatMax, flowiatStd, fiatMean, fiatMax, fiatMin, fiatTotal, biatMean, biatMax, biatMin, biatStd, biatTotal, activeMax, activeMin, activeTotal, idleMean, idleMax, idleMin, fwdPN, bwdPN, fwdPLMax, fwdPLStd, bwdPLTotal, flowPLStd, flowPLTotal

The remaining 21 features are duration, flowiatMean, flowiatMin, fiatStd, activeMean, activeStd, idleStd, idleTotal, bs, ps, flowPN, fwdPLMean, fwdPLMin, fwdPLTotal, bwdPLMean, bwdPLMax, bwdPLMin, bwdPLStd, flowPLMean, flowPLMax, flowPLMin

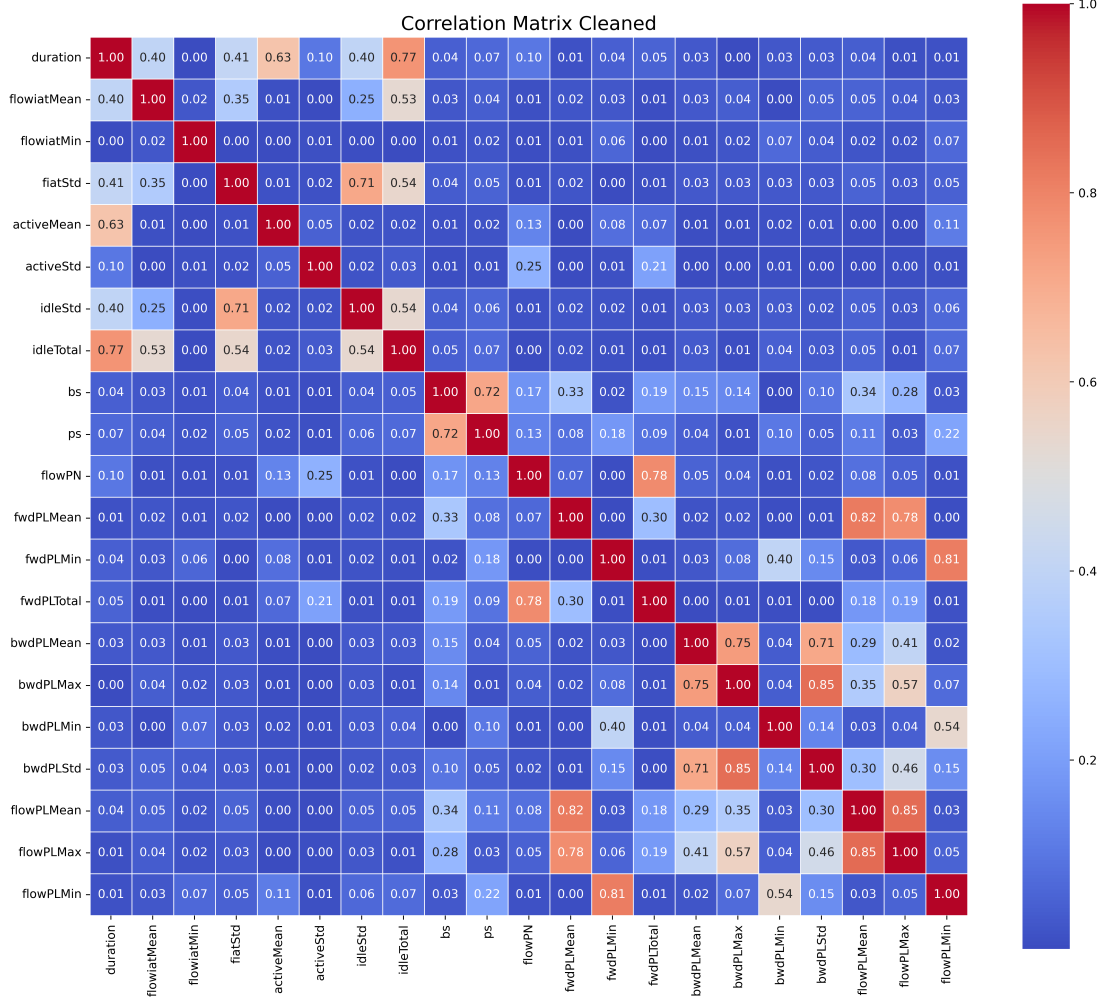


Figure 3: Correlation matrix after feature pruning.

As seen in Figure 3, the cleaned feature set no longer shows excessive dependence while still preserving the key statistics needed for accurate classification.

**Note** The correlation filter illustrated above was computed on the *entire* dataset. When the same filter is recomputed during the training (i.e., on the 80 % training split only), the exact list of retained features may differ.

## 4 Tree Models

A machine learning model based on decision trees is used for the classification task. This choice is made because decision trees work well with tabular data, such as the dataset used in this work, and provide efficient training performance. The model is based on the eXtreme Gradient Boosting (XGBoost) algorithm [7, 11], an efficient and scalable ensemble of gradient boosted decision trees that delivers state-of-the-art performance in both research and industry. Before illustrating the algorithm, the basic concepts and components on which it is based will be explored.

### 4.1 Decision Trees

Decision trees are a family of machine learning models that split the data step by step. At each internal node, the algorithm picks the feature that best separates the classes and makes the split. Splitting continues until a stopping rule is met, and the final nodes (leaves) hold the predicted class label or its probability.

#### 4.1.1 Classification and Regression Tree (CART)

Classification and Regression Tree (CART) algorithm [12] grows a binary tree by repeatedly dividing the data into two child nodes according to the information gain produced by splitting a feature at a chosen threshold. CART can be applied to both classification and regression, we will focus here on the classification case.

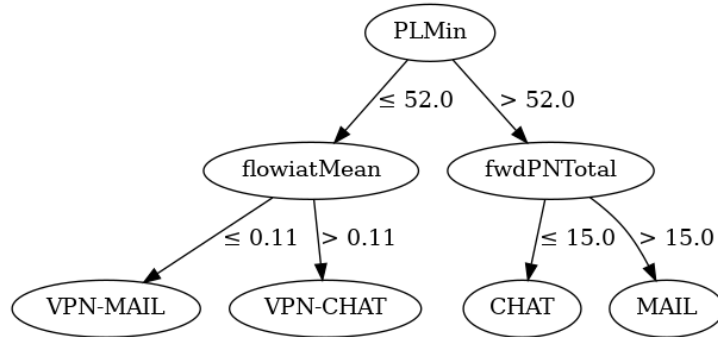


Figure 4: Cart Example

The main steps of the CART algorithm are as follows:

**Split criterion** Each split creates exactly two child nodes. The level of mixing in the data at a node is measured using the Gini impurity (Appendix A.4). While entropy could be used instead, Gini is chosen because it is faster to compute. The impurity after the split is also measured, and the difference between the two provides the information gain (Appendix A.5). This step is performed on each feature, selecting the feature-threshold pair that maximizes information gain. After dividing the data according to this optimal split, the procedure is applied recursively to each child node until the stopping criterion is met.

**Node creation** Once the best feature and threshold are found, the data is split into two child nodes. The left child node contains all samples with feature values less than or equal to the threshold, while the right child node contains all samples with feature values greater than the threshold. The feature used for the split remains available for subsequent splits.

**Stopping criterion** Different stopping criteria may be used, such as:

- All samples in a node share the same class.
- No further features are available for splitting.
- No further samples are available for splitting.
- A predefined maximum depth is reached.
- No split gives a gain higher than zero.

**Leaf creation** When the stopping criteria are met the node is converted into a leaf node. The leaf is the node that will be used for the prediction. The value predicted by the leaf is the most common class among the samples in that node.

#### 4.1.2 CART Results

When the CART algorithm is applied to the previously constructed dataset, without the limit of a max depth, the metrics results (Appendix A.6) are the following:

Table 4: Per-class metrics for the CART model. Precision (**Prec.**), recall (**Rec.**) and F1-score are shown together with the number of test samples (**Samples**).

| Class                   | Prec.      | Rec. | F1   | Samples |
|-------------------------|------------|------|------|---------|
| C2                      | 0.98       | 0.96 | 0.97 | 124     |
| CHAT                    | 0.81       | 0.86 | 0.84 | 317     |
| FT                      | 0.79       | 0.76 | 0.77 | 566     |
| MAIL                    | 0.77       | 0.82 | 0.79 | 73      |
| VIDEO_CALL              | 0.55       | 0.59 | 0.57 | 156     |
| VIDEO_STREAMING         | 0.91       | 0.91 | 0.91 | 535     |
| VOIP                    | 0.67       | 0.64 | 0.66 | 420     |
| VPN-CHAT                | 0.62       | 0.67 | 0.64 | 66      |
| VPN-FT                  | 0.84       | 0.80 | 0.82 | 160     |
| VPN-MAIL                | 0.81       | 0.86 | 0.83 | 29      |
| VPN-P2P                 | 0.91       | 0.94 | 0.92 | 65      |
| VPN-VIDEO_STREAMING     | 0.85       | 0.89 | 0.87 | 84      |
| VPN-VOIP                | 0.90       | 0.89 | 0.90 | 209     |
| <b>Macro avg.</b>       | 0.80       | 0.82 | 0.81 | 2804    |
| <b>Weighted avg.</b>    | 0.80       | 0.80 | 0.80 | 2804    |
| <b>Overall accuracy</b> | 0.8024     |      |      |         |
| <b>Training time</b>    | 43m 54.95s |      |      |         |

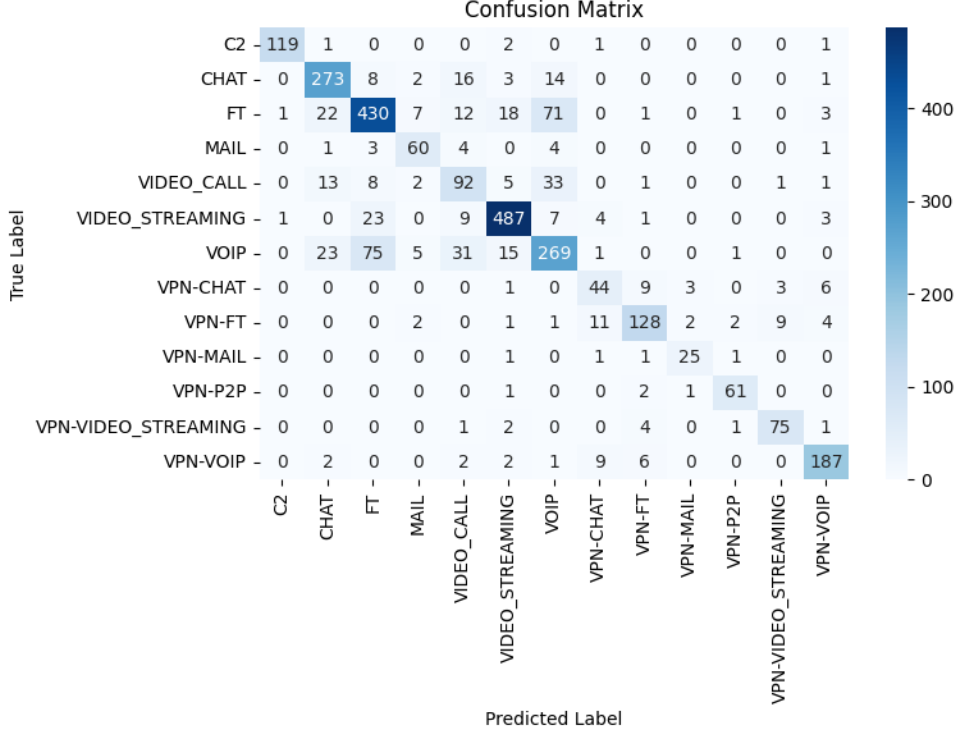


Figure 5: Confusion Matrix

The classifier achieves an overall accuracy of **0.8024** on 2 804 test flows. Classes such as C2, VIDEO\_STREAMING, and VPN-P2P show the best performance, with F1-scores of 0.91 or higher. The hardest classes are VIDEO\_CALL and VPN-CHAT, whose F1-scores fall below 0.65. The macro-averaged F1-score of 0.81 indicates that, despite the uneven class sizes, the model performs fairly evenly across classes.

## 4.2 Tree Ensembles

### 4.2.1 Random Forests

A single decision tree may struggle on complex datasets. A common fix is to grow many trees, train each one in a slightly different way, and then combine their predictions (for classification, the final label is the class with the most votes). Because each tree views the data from a different angle and learns different patterns, the ensemble is usually more accurate and less prone to overfitting.

The simplest tree ensemble is the *Random Forest*, where each tree is trained on a bootstrap sample: a set of training records, the same size as the original set, drawn *with* replacement, so some rows repeat and others are left out. On average, a tree sees only about 63 % of the unique instances. At every split, the tree may choose from only a random subset of the features, usually  $\sqrt{f}$  features for classification, where  $f$  is the total number of features. This random selection of rows and columns makes the trees less similar and lowers overfitting.



#### 4.2.2 Random Forests Results

When the Random Forest is applied to the previously constructed dataset, without the limit of a max depth and 100 trees, the metrics results are the following:

Table 5: Per-class metrics for the Random Forest model. Precision (**Prec.**), recall (**Rec.**) and F1-score are shown together with the number of test samples (**Samples**).

| Class                   | Prec.         | Rec. | F1   | Samples |
|-------------------------|---------------|------|------|---------|
| C2                      | 0.94          | 0.94 | 0.94 | 124     |
| CHAT                    | 0.82          | 0.87 | 0.85 | 317     |
| FT                      | 0.75          | 0.73 | 0.74 | 566     |
| MAIL                    | 0.71          | 0.74 | 0.72 | 73      |
| VIDEO_CALL              | 0.52          | 0.51 | 0.51 | 156     |
| VIDEO_STREAMING         | 0.82          | 0.84 | 0.83 | 535     |
| VOIP                    | 0.62          | 0.61 | 0.62 | 420     |
| VPN-CHAT                | 0.71          | 0.59 | 0.64 | 66      |
| VPN-FT                  | 0.74          | 0.77 | 0.75 | 160     |
| VPN-MAIL                | 0.93          | 0.86 | 0.89 | 29      |
| VPN-P2P                 | 0.88          | 0.78 | 0.83 | 65      |
| VPN-VIDEO_STREAMING     | 0.74          | 0.80 | 0.77 | 84      |
| VPN-VOIP                | 0.77          | 0.76 | 0.77 | 209     |
| <b>Macro avg.</b>       | 0.76          | 0.75 | 0.76 | –       |
| <b>Weighted avg.</b>    | 0.75          | 0.75 | 0.75 | –       |
| <b>Overall accuracy</b> | 0.7514        |      |      |         |
| <b>Training time</b>    | 27h 37m 3.37s |      |      |         |

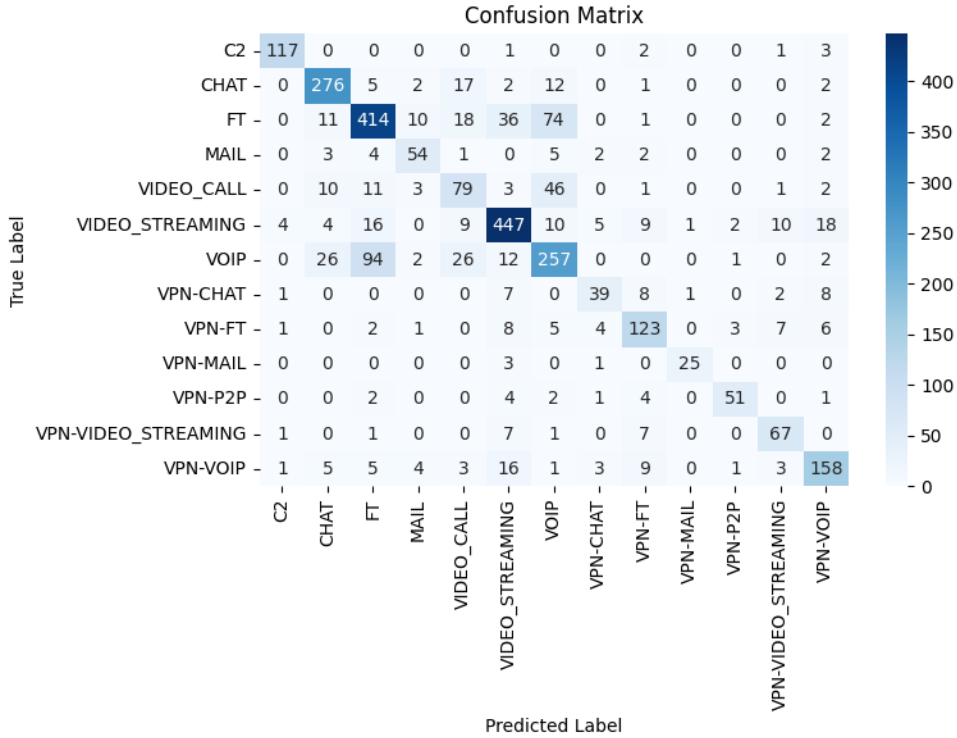


Figure 6: Confusion Matrix



The model reaches an overall accuracy of **0.7514** on 2 804 test flows. The best F1-scores are obtained by C2 (0.94), VPN-MAIL (0.89), and VIDEO\_STREAMING (0.83). VIDEO\_CALL (0.51) and VPN-CHAT (0.64) remain the hardest classes. The macro-averaged F1-score is 0.76, showing that performance is fairly balanced across classes even with different class sizes<sup>3</sup>.

## 4.3 Boosting

Another way to build an ensemble of trees is boosting, where each tree is trained to correct the errors of its predecessor. For example, if we have ten samples and the first tree classifies seven correctly, the second tree will pay particular attention to the remaining three misclassified samples. Boosting is applied starting with a simpler version of the dataset, which contains only a binary variable for the class: 0 (non-VPN) and 1 (VPN). The CART algorithm also needs to be adapted so that it works with boosting.

### 4.3.1 Binary Boosting Algorithm

Each tree outputs a *score* (a real number). The scores from all trees are summed and passed through the sigmoid activation function (Appendix A.7)  $p = \sigma(\text{score})$  to obtain the probability that the label is True ( $p > 0.5$ ).

**Training** Training proceeds in rounds, using the *entire* training set at each iteration:

1. **Initialisation** A vector  $f$  of size  $n$  (where  $n$  is the number of training samples) is initialized to 0, corresponding to a probability of 0.5 after activation. This vector holds the *scores* of the ensemble.
2. **Gradient and Hessian computation** For each sample  $i$  the gradient  $g_i$  (Appendix A.9) and Hessian  $h_i$  (Appendix A.10) of the loss function with respect to the current prediction  $f_i$  are computed.
3. **CART tree construction** Using  $g_i$  and  $h_i$ , a CART tree is constructed in a modified manner than the standard one, as detailed in Section 4.3.2.
4. **Prediction update** The vector  $f$  is updated with the scores of the new tree on each training sample:
$$f_i \leftarrow f_i + \text{leaf\_value.}$$
5. **Next round** The gradient  $g_i$  and Hessian  $h_i$  are recomputed with the updated  $f_i$  values, and steps 3-4 are repeated.

The procedure stops after a fixed number of rounds or when the training or validation loss ceases to improve.

Once training is completed, the model can be used to make predictions by summing all the scores of the trees on a new sample and applying the sigmoid function to the result. The resulting probability is then compared to a threshold of 0.5, if the value exceeds 0.5, the sample is classified as VPN, otherwise it is classified as Non-VPN.

---

<sup>3</sup>The results could probably be improved by tuning the model’s hyperparameters, such as the number of trees and the maximum tree depth.

### 4.3.2 Boosting CART Tree Construction

The CART tree construction is similar to the standard one, but with some differences. The main steps are as follows:

**Split criterion** Boosting trees do not rely on the Gini impurity like standard CART. At each candidate split the gain will be how much the new tree would *reduce the loss* of the current ensemble (Appendix A.8). The tree structure is explored as usual, but the information gain is now computed from the gradients and Hessians of the loss:

$$\text{Gain} = \frac{1}{2} \left( \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right) - \gamma,$$

where

$G_L, H_L$  gradient and Hessian for the left child,

$G_R, H_R$  gradient and Hessian for the right child,

$G, H$  gradient and Hessian for the parent node,

$\lambda$   $L_2$  regularisation term,

$\gamma$  minimum loss reduction required to split a leaf.

A split is accepted only if  $\text{Gain} > 0$ ; otherwise the node remains a leaf.

**Node creation** After selecting the best feature and threshold, the data are divided into left ( $\leq$  threshold) and right ( $>$  threshold) child nodes, exactly as in CART. The splitting feature can still be reused deeper in the tree.

**Stopping criterion** The stopping criteria are the same as the standard CART algorithm:

- All samples in a node share the same class.
- No further features are available for splitting.
- No further samples are available for splitting.
- A predefined maximum depth is reached.
- No split gives a gain higher than zero.

**Leaf creation** Unlike a standard CART tree, a boosting tree does not store the most common class in each leaf. Instead, every leaf holds a score that later contributes to the class probability. For a given leaf, the value is

$$\text{score} = \eta - \frac{G}{H + \lambda_2}$$

where  $G$  and  $H$  are the summed first and second order gradients of the loss within the leaf,  $\lambda_2$  is the  $L_2$  regularisation term, and  $\eta$  is the learning rate. A small  $\eta$  makes each tree's contribution modest, yielding a more conservative model, whereas a large  $\eta$  speeds up learning but may cause overfitting.

### 4.3.3 Binary Boosting Results

When the Binary Boosting is applied to the simplified dataset with only VPN (1) and Non-VPN (0) classes, without the limit of a max depth and 100 trees, the metrics results are the following

Table 6: Per-class metrics for the Binary Boosting model. Precision (**Prec.**), recall (**Rec.**) and F1-score are shown together with the number of test samples (**Samples**).

| Class                   | Prec.        | Rec. | F1   | Samples |
|-------------------------|--------------|------|------|---------|
| 0                       | 1.00         | 1.00 | 1.00 | 2 191   |
| 1                       | 0.99         | 0.99 | 0.99 | 613     |
| <b>Macro avg.</b>       | 0.99         | 0.99 | 0.99 | —       |
| <b>Weighted avg.</b>    | 1.00         | 1.00 | 1.00 | —       |
| <b>Overall accuracy</b> | 0.9954       |      |      |         |
| <b>Training time</b>    | 1h 9m 18.60s |      |      |         |

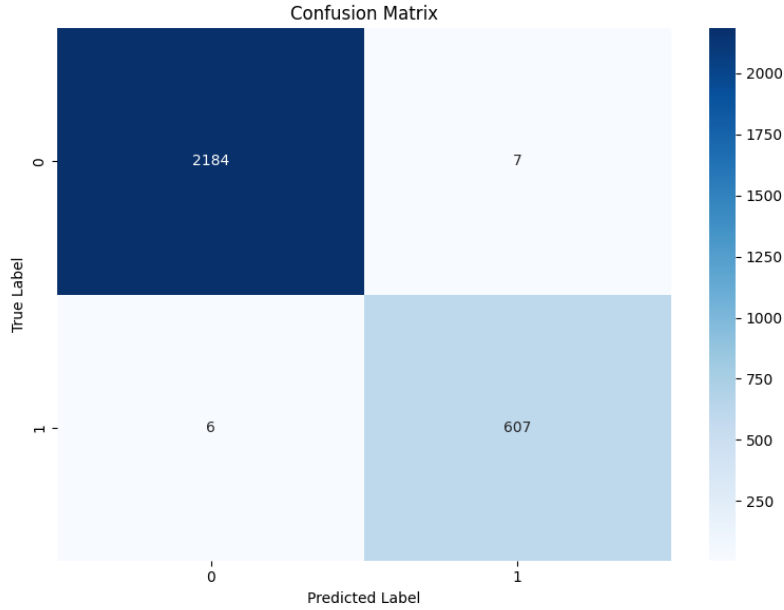


Figure 7: Confusion Matrix

The model achieves an overall accuracy of **0.9954** on 2 804 test samples. Both classes are classified almost perfectly, with F1-scores of 1.00 and 0.99, respectively. The high macro-averaged F1-score (0.99) confirms that performance is balanced across the two classes.

## 4.4 Multi-Class Boosting

### 4.4.1 Multi-Class Boosting Algorithm

To extend boosting from two to  $C$  classes, we create a binary label for every class  $c \in \{1, \dots, C\}$

$$y^{(c)} = \begin{cases} 1, & \text{if the true class is } c, \\ 0, & \text{otherwise,} \end{cases}$$

and train an ensemble of  $T$  trees to separate class  $c$  from the rest. If the binary model used  $T = 10$  trees, the multi-class version uses  $C \times T$  trees (e.g. 30 trees when  $C = 3$ ). Each tree is grown exactly as in the binary case, with the same split rule, stopping criteria, and leaf value formula. However, gradients and Hessians are computed from an activated prediction that takes in account the results for the other classes, which is the softmax function (Appendix A.11):

**Prediction** After all  $C$  ensembles are fitted, every test sample receives  $C$  raw scores  $\{s^{(1)}, \dots, s^{(C)}\}$  (by summing the leaf values along each tree path). These scores are converted to class probabilities with the soft-max function. The predicted label is the class with the largest probability,  $\hat{y} = \arg \max_c p^{(c)}$ .

This approach keeps the training logic almost identical to the binary case while allowing the model to output a proper probability distribution over all classes.

#### 4.4.2 Multi-Class Boosting Results

When the Multi Class Boosting is applied to the previously constructed dataset, without the limit of a max depth and 100 trees per class, the metrics results are the following

Table 7: Per-class metrics for the boosting model. Precision (**Prec.**), recall (**Rec.**) and F1-score are shown together with the number of test samples (**Samples**).

| Class                   | Prec.         | Rec. | F1   | Samples |
|-------------------------|---------------|------|------|---------|
| C2                      | 0.99          | 0.98 | 0.99 | 124     |
| CHAT                    | 0.89          | 0.90 | 0.89 | 317     |
| FT                      | 0.81          | 0.79 | 0.80 | 566     |
| MAIL                    | 0.87          | 0.82 | 0.85 | 73      |
| VIDEO_CALL              | 0.62          | 0.54 | 0.58 | 156     |
| VIDEO_STREAMING         | 0.93          | 0.96 | 0.94 | 535     |
| VOIP                    | 0.68          | 0.72 | 0.70 | 420     |
| VPN-CHAT                | 0.79          | 0.74 | 0.77 | 66      |
| VPN-FT                  | 0.85          | 0.88 | 0.87 | 160     |
| VPN-MAIL                | 0.96          | 0.93 | 0.95 | 29      |
| VPN-P2P                 | 0.98          | 0.92 | 0.95 | 65      |
| VPN-VIDEO_STREAMING     | 0.90          | 0.95 | 0.92 | 84      |
| VPN-VOIP                | 0.95          | 0.91 | 0.93 | 209     |
| <b>Macro avg.</b>       | 0.86          | 0.85 | 0.86 | —       |
| <b>Weighted avg.</b>    | 0.84          | 0.84 | 0.84 | —       |
| <b>Overall accuracy</b> | 0.8424        |      |      |         |
| <b>Training time</b>    | 27h 3m 50.54s |      |      |         |

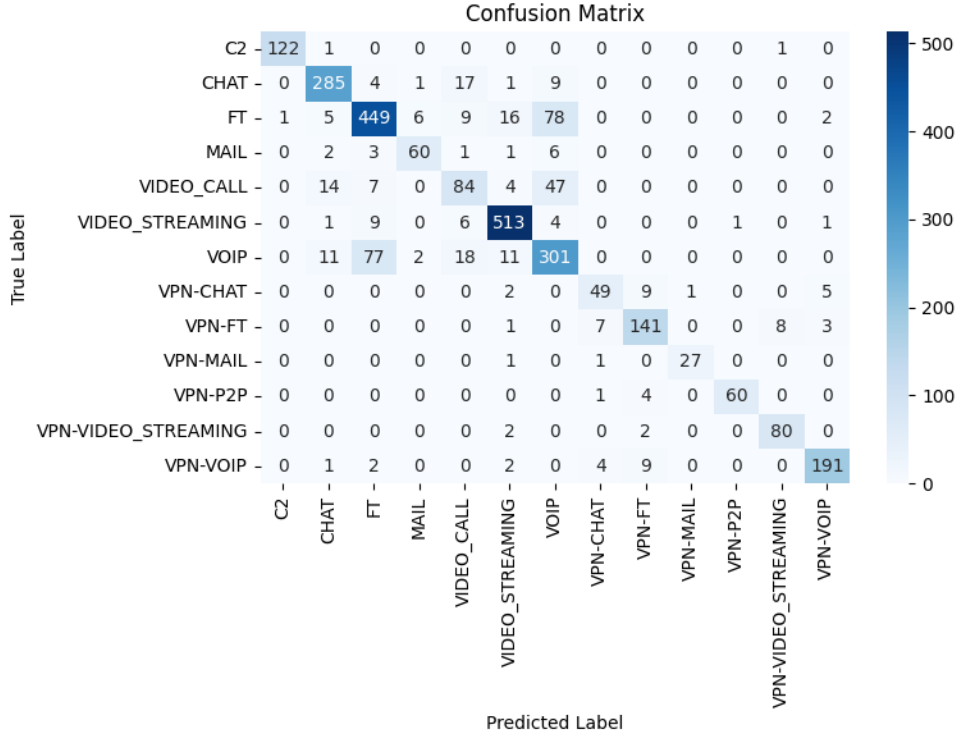


Figure 8: Confusion Matrix

The model attains an overall accuracy of **0.8424** on 2804 test flows. Top performers include C2, VIDEO\_STREAMING, and VPN-P2P, each achieving F1-scores above 0.94. VIDEO\_CALL remains the hardest class ( $F1 = 0.58$ ), followed by VOIP. The macro-averaged F1-score of 0.86 shows that the model maintains strong, fairly balanced performance across all classes despite the class size differences.

## 5 Conclusion

The data acquisition and preprocessing steps show how much information can be extracted from encrypted traffic, even when the payload is unavailable. Although only two raw measurements, packet size and arrival time, were captured for each packet, their distributions carried diverse information: after the correlation filter, twenty-one distinct features remained.

Tree-based models proved effective for classification. On the simplified VPN vs non-VPN task (Table 6), the binary boosting model achieved an accuracy of 0.9954, confirming that VPN flows leave a clear sign in timing and size metadata.

For the full multi-class task (Table 7), the boosting model reached an overall accuracy of 0.8424. Performance, however, varied by class: only 70% of true VOIP samples were correctly labelled. Interestingly, the corresponding VPN flows (labeled by VPN-VOIP) were classified much more reliably ( $F1 = 0.93$ ). This suggests that VPN tunnelling not only leaves a stronger metadata trace, like shown by the binary model results, but also enable an easier classification of the underlying application.

These outcomes underline that even comparatively simple models can exploit exposed metadata, packet timing and size, to infer application behavior.

In a defensive setting, where attackers rely on similar machine-learning analyzes, counter-measures such as packet padding can mask size clues. Altering timing, by contrast, could be more difficult for real-time services like VoIP, where even small delays degrade the quality of the service.

**Future Directions and Practical Considerations** Further improvements could be achieved through systematic hyper-parameter tuning, such as adjusting the learning rate, maximum tree depth, number of trees, and regularization parameters, to enhance model generalization and performance. In addition, the models should be tested on live, real-time traffic to evaluate their robustness under practical network conditions, including unpredictable noise, background traffic, and unseen applications.

Optimizing the efficiency of the full pipeline is also essential for deployment in real-world environments. This includes reducing the computational cost of feature extraction and model inference. One strategy could be re-implementing the most performance-critical parts of the system in a lower-level language like C or C++, or integrating GPU acceleration for parallel processing, especially during training and inference stages.

Lastly, the work could take a more security-oriented direction by repurposing these models for anomaly detection. Instead of only assigning class labels, the models could be trained to identify statistical outliers or behavior that deviates from known benign traffic patterns. This would enable the detection of previously unseen attacks or malicious communications—such as covert command-and-control traffic—without requiring decryption of the packet contents, preserving user privacy while enhancing network security.

## A Appendix

### A.1 .pcap and .pcapng

The .pcap and .pcapng (PCAP Next Generation) formats are used to store network packet captures, including timestamps, packet sizes, protocol headers, protocol layers, and payload data. The .pcapng format is an enhanced successor to .pcap, providing a more flexible, extensible file structure and support for additional metadata fields.

### A.2 TCP RTT Calculation

The round-trip time (RTT) is the interval between sending a TCP segment and receiving its corresponding acknowledgement. TCP uses the RTT estimate both to adjust its congestion window and to set its retransmission timeout. Although RTT varies over time, it typically oscillates around a slowly changing average.

RTT is estimated using an exponentially weighted moving average. Starting from an initial value  $RTT_0$ , each time a segment is sent, we record the measured sample RTT  $M_i$ . We then update our estimate as

$$RTT_{i+1} = \alpha RTT_i + (1 - \alpha) M_i,$$

where  $\alpha$  is the smoothing factor (in this work,  $\alpha = \frac{7}{8}$ ).

### A.3 Correlation Matrix

Correlation matrix store the Pearson correlation coefficient between the features, let be the dataset a matrix with n lines and m columns, with each column that represents a feature

$$\begin{bmatrix} f_{1,1} & f_{1,2} & \cdots & f_{1,m} \\ f_{2,1} & f_{2,2} & \cdots & f_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ f_{n,1} & f_{n,2} & \cdots & f_{n,m} \end{bmatrix} \xrightarrow{\text{Correlation Matrix}} \begin{bmatrix} \rho_{f_1,f_1} & \rho_{f_1,f_2} & \cdots & \rho_{f_1,f_n} \\ \rho_{f_2,f_1} & \rho_{f_2,f_2} & \cdots & \rho_{f_2,f_n} \\ \vdots & \vdots & \ddots & \vdots \\ \rho_{f_n,f_1} & \rho_{f_n,f_2} & \cdots & \rho_{f_n,f_n} \end{bmatrix}$$

$$\rho_{f_i,f_j} = \frac{\text{cov}(f_i, f_j)}{\sigma_{f_i} \sigma_{f_j}}$$

$$\sigma_X = \sqrt{\text{Var}(X)}$$

$$\sigma_Y = \sqrt{\text{Var}(Y)}$$

$$\text{cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])].$$

$$\rho_{X,Y} = \frac{\mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]}{\sqrt{\mathbb{E}[(X - \mathbb{E}[X])^2]} \sqrt{\mathbb{E}[(Y - \mathbb{E}[Y])^2]}}$$

The correlation coefficients satisfy for all  $i, j$ :  $-1 \leq \rho_{f_i,f_j} \leq 1$ ,  $\rho_{f_i,f_j} = \rho_{f_j,f_i}$ , and  $\rho_{f_i,f_j} = 1 \iff f_i$  and  $f_j$  have the same elements

The correlation coefficient says at which point two set of data increase or decrease simultaneously.

## A.4 Gini impurity

In classification tasks, Gini impurity quantifies how mixed the class labels are within a subset. It attains its minimum value of 0 when all samples belong to the same class (perfect purity), and its maximum value of  $1 - \frac{1}{C}$  when the samples are evenly split among the  $C$  classes (maximum impurity). Formally,

$$\text{Gini} = 1 - \sum_{i=1}^C p_i^2,$$

where

$$C = \text{number of classes}, \quad p_i = \frac{\text{count of samples in class } i}{\text{total sample count}}.$$

## A.5 Information Gain with Weighted Gini

When a node is split at a threshold  $t$  in a decision tree, the reduction in Gini impurity is measured. This reduction represents the information gain:

$$\text{Gain}(t) = G_{\text{parent}} - \left( \frac{N_l}{N} G_l + \frac{N_r}{N} G_r \right),$$

where:

$N_l :=$  number of samples with the feature value  $\leq t$ ,  $G_l :=$  Gini impurity of the left subset,  
 $N_r :=$  number of samples with the feature value  $> t$ ,  $G_r :=$  Gini impurity of the right subset,  
 $N :=$  the total number of samples in the parent node,  $G_{\text{parent}} :=$  Gini impurity before the split.

An alternative to maximizing this gain is to select the  $t$  that minimizes the weighted average impurity:

$$\frac{N_l}{N} G_l + \frac{N_r}{N} G_r.$$

Instead, we use the  $\text{Gain}(t)$  expression to maintain consistency with the code and the other models.

## A.6 Metrics

For each class  $i$ , let:

$$\text{TP}, \text{FP}, \text{FN}, \text{TN} \quad \text{and} \quad s_i = \text{TP} + \text{FN},$$

where  $C$  is the number of classes and  $N = \sum_i s_i$  is the total number of samples. We define:

$$\begin{aligned} \text{Precision}_i &= \frac{\text{TP}}{\text{TP} + \text{FP}}, & \text{Recall}_i &= \frac{\text{TP}}{\text{TP} + \text{FN}}, \\ \text{F1}_i &= 2 \frac{\text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}, & \text{Accuracy} &= \frac{\sum_i (\text{TP} + \text{TN})}{N}. \end{aligned}$$

We also report:



$$\begin{aligned}\text{Macro-Avg(metric)} &= \frac{1}{C} \sum_{i=1}^C \text{metric}_i, \\ \text{Weighted-Avg(metric)} &= \frac{1}{N} \sum_{i=1}^C s_i \text{metric}_i.\end{aligned}$$

## A.7 Sigmoid Function

The sigmoid (logistic) function maps any real number  $f$  to the open interval  $(0, 1)$ :

$$\sigma(f) = \frac{1}{1 + e^{-f}}.$$

In binary classification we interpret  $p = \sigma(f)$  as the model's estimated probability that the instance belongs to the positive class.

## A.8 Binary Cross-Entropy Loss

For a true label  $y \in \{0, 1\}$  and predicted probability of the positive class  $p = \sigma(f)$ , the binary cross-entropy (log-loss) that represents how well the model predicts the true label is defined as

$$\ell(y, p) = -[y \ln p + (1 - y) \ln(1 - p)].$$

## A.9 Gradient

The gradient of the binary cross-entropy loss with respect to the model's output  $f$  is given by

$$\ell(y, f) = -[y \ln \sigma(f) + (1 - y) \ln(1 - \sigma(f))].$$

**Differentiate using the chain rule.**

$$\frac{d\ell}{df} = -\left[y \frac{1}{\sigma(f)} - (1 - y) \frac{1}{1 - \sigma(f)}\right] \frac{d\sigma(f)}{df}.$$

**Compute the derivative of  $\sigma(f)$ .**

$$\frac{d\sigma(f)}{df} = \sigma(f)(1 - \sigma(f)) = p(1 - p).$$

**Substitute  $p$  and simplify.**

$$\frac{d\ell}{df} = -\left[y \frac{1}{p} - (1 - y) \frac{1}{1 - p}\right] p(1 - p) = -[y(1 - p) - (1 - y)p] = p - y.$$

$$\boxed{\frac{\partial \ell}{\partial f} = p - y.}$$

## A.10 Hessian

The Hessian of the binary cross-entropy loss is the second derivative of the loss with respect to the model's output  $f$ .

$$\frac{\partial^2 \ell}{\partial f^2} = \frac{d}{df}(p - y) = \frac{dp}{df} = p(1 - p).$$

## A.11 Softmax Function

The softmax function is used to convert a vector of real numbers into a probability distribution over multiple classes. Given a vector  $\mathbf{z} = [z_1, z_2, \dots, z_K]$  with  $K$  elements, the softmax function is defined as:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, 2, \dots, K$$

where  $e$  is the base of the natural logarithm, and  $z_i$  is the  $i$ -th element of the input vector  $\mathbf{z}$ . The output of the softmax function is a vector of probabilities that sum to 1, where each element represents the probability of the corresponding class.

## References

- [1] Gerard Draper-Gil et al. “Characterization of Encrypted and VPN Traffic using Time-related Features”. In: *Proceedings of the 2nd International Conference on Information Systems Security and Privacy - ICISSP*. INSTICC. SciTePress, 2016, pp. 407–414. ISBN: 978-989-758-167-0. DOI: [10.5220/0005740704070414](https://doi.org/10.5220/0005740704070414).
- [2] Steven Jorgensen et al. “Extensible Machine Learning for Encrypted Network Traffic Application Labeling via Uncertainty Quantification”. In: *IEEE Transactions on Artificial Intelligence* 5.1 (2024), pp. 420–433. DOI: [10.1109/TAI.2023.3244168](https://doi.org/10.1109/TAI.2023.3244168).
- [3] Ahmad Antari et al. *Network Traffic Classification Using Machine Learning, Transformer, and Large Language Models*. 2025. arXiv: [2503.02141](https://arxiv.org/abs/2503.02141) [cs.LG]. URL: <https://arxiv.org/abs/2503.02141>.
- [4] Ting-Li Huoh et al. “Flow-Based Encrypted Network Traffic Classification With Graph Neural Networks”. In: *IEEE Transactions on Network and Service Management* 20.2 (2023), pp. 1224–1237. DOI: [10.1109/TNSM.2022.3227500](https://doi.org/10.1109/TNSM.2022.3227500).
- [5] Ahmad Azab et al. “Network traffic classification: Techniques, datasets, and challenges”. In: (Sept. 2022). DOI: [10.1016/j.dcan.2022.09.009](https://doi.org/10.1016/j.dcan.2022.09.009).
- [6] Meng Shen et al. “Machine Learning-Powered Encrypted Network Traffic Analysis: A Comprehensive Survey”. In: *IEEE Communications Surveys & Tutorials* 25.1 (2023), pp. 791–824. DOI: [10.1109/COMST.2022.3208196](https://doi.org/10.1109/COMST.2022.3208196).
- [7] Tianqi Chen and Carlos Guestrin. “XGBoost: A scalable tree boosting system”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM. 2016, pp. 785–794. DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- [8] KimiNewt. *PyShark: Network Packet Parser using TShark*. <https://github.com/KimiNewt/pyshark>. A Python wrapper for TShark, allowing packet parsing from live captures and PCAP files. 2025.
- [9] Charles R. Harris et al. “Array programming with NumPy”. In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2). URL: <https://doi.org/10.1038/s41586-020-2649-2>.
- [10] Jeff Reback et al. “pandas-dev/pandas: Pandas”. In: *Zenodo* (2020). DOI: [10.5281/zenodo.3509134](https://doi.org/10.5281/zenodo.3509134).
- [11] Tianqi Chen, Carlos Guestrin, and contributors. *XGBoost: Scalable, Portable and Distributed Gradient Boosting*. <https://github.com/dmlc/xgboost>. Accessed: 2025-05-12. 2016.
- [12] Leo Breiman et al. *Classification and Regression Trees*. Belmont, California: Wadsworth International Group, 1984.